Expert Systems With Applications 254 (2024) 124427

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

# Comparing Natural Language Processing and Quantum Natural Processing approaches in text classification tasks

David Peral-García [a,*], Juan Cruz-Benito [b], Francisco José García-Peñalvo [c]

[a] Expert Systems and Applications Laboratory - ESALAB, Faculty of Science, Universidad de Salamanca, Plaza de los Caídos s/n, 37008 Salamanca, Spain[1]
[b] IBM Quantum, IBM T.J. Watson Research Center, Yorktown Heights, NY 10598, USA
[c] GRIAL Research Group, Department of Computers and Automatics, Research Institute for Educational Sciences, Universidad de Salamanca, Paseo de Canalejas, 169, Salamanca 37008, Spain[1]

ARTICLE INFO

*Keywords:*
Quantum computing
Quantum Machine Learning
Quantum natural language processing

ABSTRACT

Quantum physics and mechanics have demonstrated significant advances and promising results in different areas using the current near-term devices. One emerging subarea in quantum machine learning is quantum natural language processing, which combines quantum computing advantages and speedups with language processing algorithms to create and perform natural language tasks such as text classification or generation. The libraries and toolboxes used in this subarea include DisCoPy and lambeq, which are used to transform sentences into string diagrams or monoidal functors, convert these diagrams into quantum circuits or *ansatz* and embed it into a quantum model. In this study, we used both libraries with different text-based datasets to perform sentiment analysis via classification. To do so, we create synthetic datasets to train the different models. After we obtain satisfactory results, we test the resulting models with known datasets. Despite its promising results, quantum natural language processing is far from achieving its full potential. To achieve this potential, the quantum software and hardware must be improved to make them suitable for use with more extensive and complex datasets and other tasks.

## 1. Introduction

Quantum mechanics algorithms have been present since the late 1990s. For example, Shor's algorithm factorizes integers in polynomial time instead of superpolynomial time (Shor, 1997) or Grover's search algorithm (Grover, 1996). However, quantum devices have been available only since the past few years and are usable only when these algorithms could be implemented and applied.

The current quantum devices, called near-term devices, have made it possible to perform quantum-related tasks and implement quantum theoretical algorithms in quantum real systems and simulators. Although, nowadays, there are examples of projects developed in real quantum remote devices, these systems are already accessible and provided by different companies like IBM (Gokhale, Pande, & Pramod, 2020), Rigetti (Shenoy, Sheth, Behera, & Panigrahi, 2020), Google Tensorflow Quantum Simulator (Wu et al., 2021), Amazon Braket Local Simulator (Wu et al., 2021) or the Matlab simulator (Lukac, Abdiyeva, & Kameyama, 2018).

These devices are still imperfect, as they do not have fault-tolerant qubits, and their relaxation time (T1), the time it takes for the qubit to decay from state $|1\rangle$ to $|0\rangle$, or the decoherence time (T2), the time at which the qubit loses its coherence, needs to be improved. Other factors that affect their performance are the state preparation, efficiency in applying gates, and readout results.

Even considering the existing devices are not perfect and fault tolerant, they have already allowed scientists to run quantum algorithms and obtain significant results and are paving the way to the 'quantum advantage' era. Moreover, they have already proved their efficiency in areas like chemistry (Bauer, Bravyi, Motta, & Chan, 2020), finance (Emmanoulopoulos & Dimoska, 2022), or machine learning (Biamonte, Wittek, Pancotti, Rebentrost, Wiebe et al., 2017).

After a systematic literature review of QML (García, Cruz-Benito, & García-Peñalvo, 2022), we concluded that Quantum Natural Language Processing (QNLP) is one emergent subarea. The mathematical and conceptual basis for the area is described in (Coecke, Sadrzadeh, & Clark, 2010). The authors created a mathematical framework to unify the distributional theory of meaning in terms of vector space models, and a compositional theory for grammatical types, which relies on the algebra of Pregroups, introduced by Lambek (Lambek, 1958). On one

* Corresponding author.
*E-mail addresses:* daveral@usal.es (D. Peral-García), juan.cruz.benito@ibm.com (J. Cruz-Benito), fgarcia@usal.es (F.J. García-Peñalvo).
[1] https://ror.org/02f40zc51.

https://doi.org/10.1016/j.eswa.2024.124427
Received 16 May 2023; Received in revised form 23 May 2024; Accepted 4 June 2024
Available online 6 June 2024
0957-4174/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

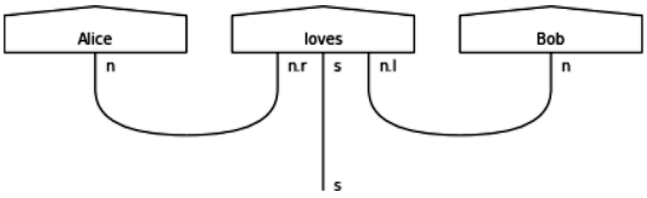



**Fig. 1.** DisCoPy syntax example.

side, vector space gives us a notion of distance between words, so the inner product can be used to determine how close in meaning one word is to another. On the other side, Pregroup grammar – which is a particularly ordered monoid – p $(P, \leq, \cdot, 1, (-)^l, (-)^r)$, whose each element $p \in P$ has a left adjoint $p^l$ and a right adjoint $p^r$- assist in constructing the meaning of the whole sentence.

One of the reasons that the authors in (Coecke, de Felice, Meichanetzidis, & Toumi, 2020) describe QNLP as 'Quantum native' is because both quantum theory and natural language use of vector spaces for describing states, this implies that natural language naturally fit on quantum hardware. Moreover, with the improvement in capabilities and reliability on quantum hardware (Ding et al., 2023; Romito, 2023; Shlosberg, Jena, Mukhopadhyay, Haase, Leditzky et al., 2023), the problem that exists with exponential increase space we have due to the tensor product on classical hardware would be efficiently handle.

In (Coecke et al., 2020) Zeng and Coecke presents how DisCoCat execution on quantum hardware would lead to quantum advantage in terms of speed-up. Also, in this paper they calculate the necessary storage for verbs in quantum and classical frameworks. They estimate for a 2K nouns corpus, with 1 transitive verb the required storage is $8 \times 10^9$ bits in classical computing against 33 qubits in quantum computing. In addition, in (Kerenidis & Prakash, 2017) demonstrated that other standard NLP tasks like access to data or singular value estimation can be benefited by quantum capabilities.

The main objective of this paper is to compare NLP and QNLP, according to accuracy results and computational time training, to verify if, with the current technology, we can achieve the same results with both the implementations and devices, classical and quantum. Moreover, another objective is to force to the limits of QNLP classification proofs of concept we made, in terms of classical/quantum computer resources, RAM in case of classical and qubits in quantum.

The paper is structured as follows. First, we present an introduction and state of the art about quantum computing and QML, focusing on QNLP and their principal articles and contributions. Second, we provide the materials and methods, explaining the datasets, algorithms, and hardware used. Third, we show the results achieved with different implementations, including some metrics like accuracy and time training. Fourth, we discuss the results. Finally, we highlight the primary outcomes and the future lines we can follow for further implementation.

## 2. State of the art

After the mathematical foundations are established, a group of intuitive tools is required to access and use all this theory. This is why the creation of two essential programming libraries began, DisCoPy (de Felice, Toumi, & Coecke, 2021; Toumi, de Felice, & Yeung, 2022) and lambeq (Kartsaklis et al., 2021). DisCoPy provides an intuitive syntax for defining string diagrams – which are graphical calculus for monoidal categories – and monoidal functors. DisCoPy was the tool that allowed us to perform natural language processing on quantum hardware for the first time. It provides an intuitive Python syntax for diagrams and includes methods for diagram composition, drawing, rewriting, and evaluation, allowing one to visualize and reason about computation at a high level of abstraction. Also, this library implements categorical quantum mechanics interfacing with other libraries like Tket (https://github.com/CQCL/tket for circuit compilation, PyZX (Kissinger & van de Wetering, 2020) for optimization – which implements the theory of ZX-calculus – and PennyLane. In addition, different formal grammars are implemented, including pregroup grammar [Fig. 1], cited before.

These technologies are quantum native, which allows for example, the equivalence between a ZX-diagram and a quantum circuit. The reason we select this methodology and framework is that allows us to work with a diagram based interface and leverage quantum advantages. This diagram based interface allow us to convert sentences in string diagrams, and string diagrams into ansatz, supporting us with visual representation with facilitates the creation of the different models. Regarding quantum advantages, with quantum computing we can increase exponentially the number of words which can be encode in quantum bits against the ones we can encode in classical bits.

In addition, lambeq library provides the tools to transform sentences into string diagrams, rewrite them, parameterize them, and load them into a neural network to train a model [Fig. 2].

First, a 'reader' translate the sentence into a string diagram using a particular compositional model. The library provides various models, such as the "Bag of words" model, which does not use any syntactic information, or DisCoCat, which is a syntax-based model accessible from the DisCoPy library. Moreover, the word-sequence model can be used in words composed from left to right, and three readers can interpret the sentences as a series of compositions.

Second, the string diagrams should be rewritten after converting the sentence to optimize the future transformation and training process. The optimization process can include text-cleaning tasks such as the removal of auxiliary verbs, connectors, or determiners.

Third, the string diagram is converted into a quantum circuit or tensor network based on a specific parameterization scheme and different choices of *ansatz*. The available *ansatz* in lambeq are Instantaneous Quantum Polynomial (IQP) (Havlíček et al., 2019), Sim14 (Sim, Johnson, & Aspuru-Guzik, 2019), Sim15 (Sim et al., 2019) and Strong Entangling (Schuld, Bocharov, Svore, & Wiebe, 2020).

The IQP *ansatz* [Fig. 3] interleaves layers of Hadamard gates with diagonal unitaries. The lambeq class uses $nlayers - 1$ adjacent CRz gates to implement each diagonal unitary; this *ansatz* is presented in https://cqcl.github.io/lambeq/root-api.html.

Another *ansatz* is Sim14 [Fig. 4], which is a modification of the circuit 14 presented in (Sim et al., 2019). It replaces circuit-block construction with two rings of CRx gates in opposite orientations [Fig. 7(a)].

Also, Sim15 [Fig. 5] is presented in (Sim et al., 2019) too, which arise from circuit 15. It uses two-qubit gates in a ring topology. Sim14 and Sim15 have a linear dependence with respect to the number of qubits used for data classification [Fig. 7(b)].

To train a classifier that captures short and long-range correlations in the input data, in (Schuld et al., 2020), the authors proposed the Strong Entangling *ansatz* [Fig. 6]. Which uses three single qubit rotations ($Rz$, $Ry$, and $Rz$) followed by a ladder of CNOT gates with different ranges per layer. This quantum circuit uses $U_\theta$ as the core of




**Fig. 2.** Pipeline implemented by lambeq. (Kartsaklis et al., 2021).

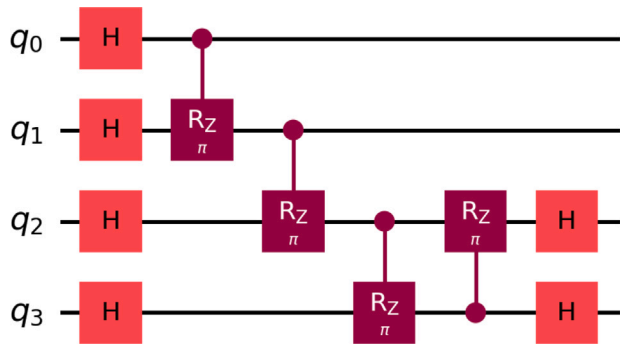



**Fig. 3.** IQP Ansatz.

the machine learning model $f(x;\theta) = y$, where $x$ is an input, $\theta$ a set of parameters, and $y$ is the prediction or output of the model.

Once the quantum circuit is processed in a quantum device, the quantum output is transformed in a classical tensor network, and the result is evaluated with a loss function. In this step, the rotation of the qubits in the circuit in the different axis is optimized to minimize the loss function. The rotation of the qubits can be modified by the controlled gates of the implemented circuits.

To train the neural net, lambeq provides different models supported by classical and quantum backends. Furthermore, NumpyModel supports the just-in-time (jit) compilation provided by the library jax, which can be used with GPU support to train the models using quantum simulation faster. In addition, however, TketModel can retrieve shot-based results from a real quantum computer. Last but not least, PytorchModel is used in classical neural networks.

Regarding the related work, some QNLP approaches can be found in (Widdows, Alexander, Zhu, Zimmerman, & Majumder, 2024) where the authors described a quantum probability approach to bigram modeling that can be applied to understand sequences of words and formal concepts. Also, in (Karamlou, Pfaffhauser, & Wootton, 2022) perform a label topic classification with a ContextFree Grammar (CFG) dataset and in (Lorenz, Pearson, Meichanetzidis, Kartsaklis, & Coecke, 2023) they use lambeq to train NLP models that solve simple sentence classification tasks on quantum hardware. None of the studies made an accuracy comparison between different datasets, labels, ansatz and layers.

## 3. Materials and methods

### 3.1. Datasets

We create various prototypical datasets for label classification tasks with a specific syntax, after that, we use an existing dataset, and we adapt it to fit into our model. A critical aspect of the sentences is the grammatical category, which is established in the prototypical datasets and modify it in the existing ones.

The restrictions regarding number of qubits available and the need of a fixed grammatical category are the main reasons we use artificial datasets instead of ordinary datasets. With this self-made datasets we can increase the required number of qubits precisely to fit in a specific quantum device. Moreover, the complexity of the ansatz is related to the type of ansatz as well as the relationships between grammatical categories, with an artificial dataset we can control and adjust this relationships in order not to increase complexity.

First, we define a list of words for each category (noun, adjective, verb, etc.). Secondly, we develop a code to create random balance datasets with the same number of samples in each class to simplify model training. There are datasets for classification between four labels in models with higher complexity and datasets with two labels in models where the number of qubits and complexity of the resulting quantum circuit are lower. The distribution for training, testing and validation is 60/20/20.

- Artificial Dataset-2 (AD2): This sentiment dataset comprises 2, 3, and 4 labels. Contains 300 records. The syntax structure is: noun + auxiliary (will/might) + verb + adjective, e.g., Igor will be somber.
- Artificial Dataset-3 (AD3): This sentiment dataset comprises 2, 3, and 4 labels. Contains 300 records. The syntax structure is: noun + conjunction + noun + verb + adjective, e.g., John and Mary are unhappy.
- Artificial Dataset-4 (AD4): This sentiment dataset comprises 2, 3, and 4 labels. Contains 300 records. The syntax structure is: noun + relative + verb + determiner + noun(relative) + verb + adjective, e.g., John who has a kid is unhappy.
- Artificial Dataset-5 (AD5): This sentiment dataset comprises 2, 3, and 4 labels. Contains 300 records. The syntax structure is: noun + relative + verb + determiner + noun + conjunction + determiner + noun + verb + adjective, e.g., John who has a kid and a plant is unhappy.
- NER Milk (MILK): This is a name entity recognition dataset comprising 2, 3, and 4 labels. It is written in the Spanish language. Contains 480 records. The syntax structure is noun + brand + adjective, noun + adjective + brand, brand + noun + adjective, brand + adjective + noun, e.g., category 1: batido Auchan entera.
- NER Kaggle (KAGG). The dataset is an adaption of https://www.kaggle.com/datasets/abhinavwalia95/entity-annotated-corpus . This is a sentiment dataset composed of 2, 3, and 4 labels. The categories are a geographical entity, time indicator, organization, and person. Contains 800 records. The syntax structure is: determiner + noun + verb + adjective, e.g., The category is London.

### 3.2. Algorithms

We propose three training approaches to compare. The first one uses classical computation, the second one simulates the quantum computation hardware with Jax library, and the third one uses real quantum devices simulators and real quantum devices.

#### 3.2.1. Classical computation

As we explained before, the first step of the lambeq approach is parsing the sentences into string diagrams. We use BobCat parser [Fig. 8] for this task. Secondly, we encoded the diagram into an *ansatz*, in this case, we use Spider *ansatz*. [Fig. 9]. Last, a Pytorch model is prepared – adjusting their parameters – and trained. An example of the code can be find in Appendix A.1.

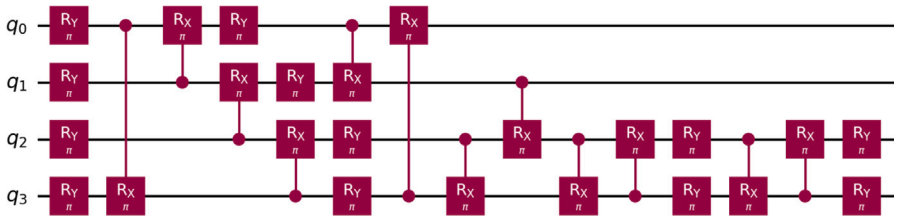



**Fig. 4.** Sim14 Ansatz.

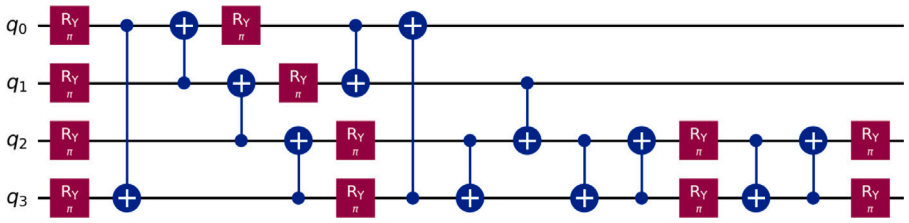


**Fig. 5.** Sim15 Ansatz.

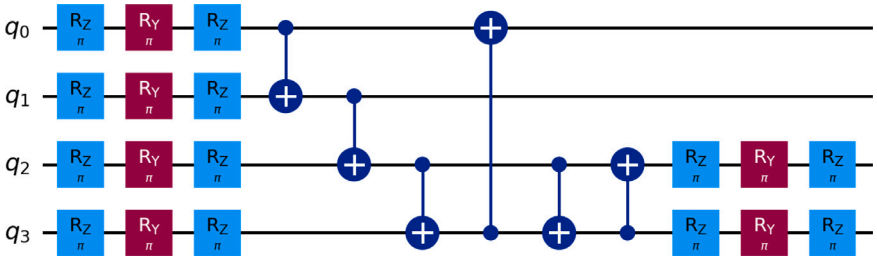
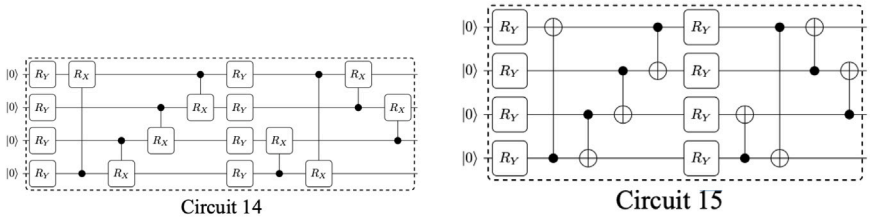


**Fig. 6.** Strong Ansatz.




(a) Circuit 14 (Sim et al., 2019). (b) Circuit 15 (Sim et al., 2019).

**Fig. 7.** Circuit 14 and Circuit 15 *ansatz*.

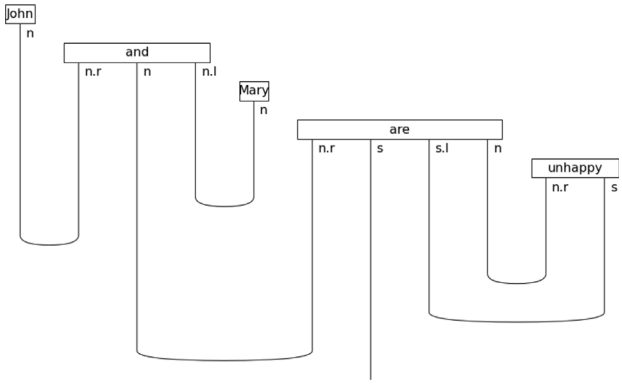



**Fig. 8.** Bobcat Parser.

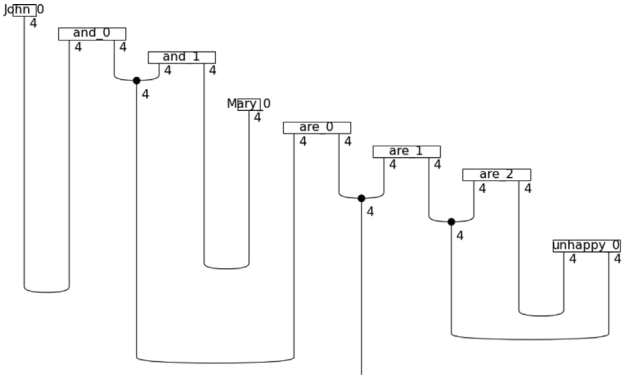



**Fig. 9.** Spider *Ansatz*.

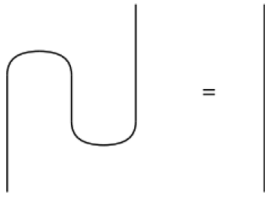



**Fig. 10.** Remove cups example.

### 3.2.2. Quantum computing

As in the previous subsection, the sentences are converted into diagrams using DisCoCat parser. After that, we removed the cups of the diagrams in order to simplify them and avoid higher computational costs. This procedure can transform the diagram in another equivalent one – which evaluation provides the same result – but reducing the number of cups and caps in the diagram, which result in a speed up of the evaluation of the diagrams [Fig. 10].

We parameterized the quantum circuit according to the *ansatz* explained in Section 2, IQP, Sim14, Sim15, and Strongy Entangled [Fig. 11]. Next, in the case of quantum simulation, a Numpy model is used together with the Jax library to reduce the training time. On the other part, to train the models in quantum real hardware we implement a Tket model to be executed in IBM quantum systems.

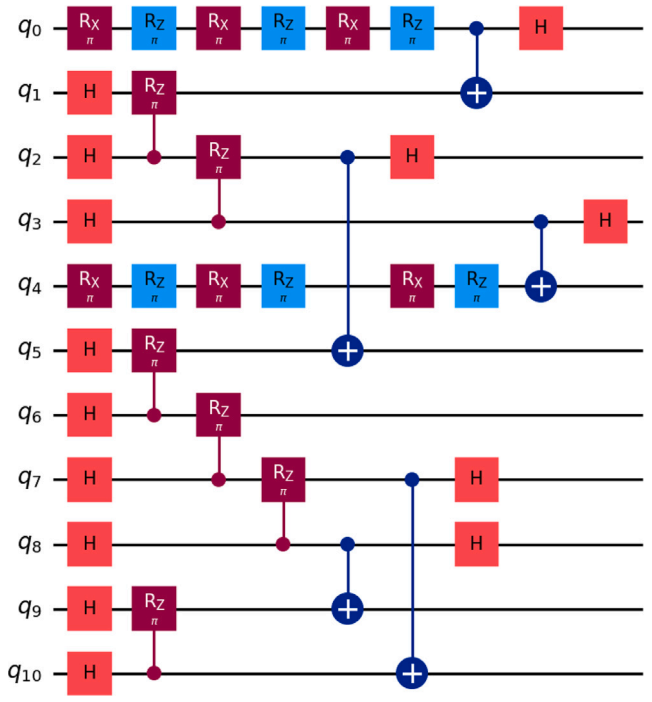



**Fig. 11.** Quantum computation with lambeq. IQP *ansatz* example.

Finally, we parameterize the model optimal hyperparameters and loss and evaluation metrics, the model is trained. Code example can be find in Appendix A.2.

### 3.3. Hardware

The classical computational system architecture used to run experiments is:

- CPU: AMD Ryzen 7 3700X, 8 Cores, 4.4 GHz
- RAM: 32 GB DDR4 3200
- GPU: NVIDIA GeForce RTX 3080 10 GB
- Operation System (OS): Ubuntu Server 18.04 LTS

The quantum devices ibm_lagos,[2] ibm_oslo[3] and ibm_nairobi[4] have the following characteristics [Table 1].

## 4. Results

We tested the previous datasets cited with lambeq, classical computing, and quantum computing. For this experiment, we adapt the different datasets to train models that classify in 2, 3, and 4 labels, to test from the most straightforward task to the most complex ones. Moreover, in the quantum simulation models, we test it with 1 layer and 2 layers of the chosen *ansatz*. For the classification of the 3 and 4 labels in the quantum simulation models, we need 2 qubits to encode the input data, which adds computational cost and complexity, that is the reason we cannot test the more complex datasets (AD4 and AD5) with the IQP-*ansatz* with 2 layers. After we had trained the models, we used the accuracy metric to test the efficiency of the models. Also, we describe the training time of the models. The results are presented in https://github.com/DaveHub5/QNLP_Classification_2_3_4_labels.

**Table 1**
Systems characteristics.

| System Name | ibm_lagos | ibm_oslo | ibm_nairobi |
|---|---|---|---|
| Qubits | 7 | 7 | 7 |
| Quantum Volume | 32 | 32 | 32 |
| Clops | 2.7K | 2.6K | 2.6K |
| Processor type | Falcon r5.11H | Falcon r5.11H | Falcon r5.11H |
| Median CNOT Error | 7.451e−3 | 8.108e−3 | 1.066e−2 |
| Median Readout Error | 1.540e−2 | 2.310e−2 | 2.630e−2 |
| Median T1 | 115.81 us | 148.32 us | 129.48 us |
| Median T2 | 77.89 us | 56.03 us | 60.37 us |

The results of each dataset are presented here [Tables 2, 3, 4]. IQ represents the quantum system having insufficient qubits to train the model, ICR that our classical computer has insufficient computational resources, and ND means that we have no data.

Moreover, we provide two more tables with the average grouped results. In [Table 8] we grouped by labels, number of layers and ansatz. Also, in [Table 9] we show the average grouped results for the 2 labels data, to compare between the classical, quantum simulator and quantum real devices.

In the time column of tab:[Table 8] we calculate the mean filtering with classical and quantum-jax to provide a fair comparison with the same amount and type of data, because the computation times of quantum real devices are really different as we can see in [Table 9].

We describe the accuracy comparing the mean of different subgroups: datasets, labels of each model, applied *ansatz* (IQP, Sim14, Sim15, Strong), the number of layers of each *ansatz* and classical and quantum hardware. Moreover, we provide the training time divided in labels as the accuracy metric [Tables 5, 6, 7].

First, in the comparison between datasets, we can observe the datasets with higher accuracy are AD3 and AD5 with 75.43% and 71.17%. Also, we have a 68.17% in AD4, 57.24% in MILK, 53.68% in AD2 and 52.03% in KAGG. We can see that NER models with 55.13% have worse accuracy than sentiment models with 67.24%.

Second, related to the number of labels of the models, the results in 2 labels models 81.96% are better than models which have to predict 3 or 4 labels, 52.83% and 50.41%.

Third, the comparative between *ansatz* (IQP, Sim14, Sim15 and Strong). We notice IQP *ansatz* have the best mean accuracy of the four *ansatz* implemented in lambeq 67.50% against Sim14 62.65%, Sim15 53.46% and Strong 61.48%. Nevertheless, Strong Entangling *ansatz* has better accuracy than IQP in the KAGG dataset, 60.04% against 50.18% of IQP.

Fourth, the models with 1 quantum layer of each *ansatz* 64.12% have greater accuracy than models with 2 layers 57.85% in every dataset.

To compare the data between classical and quantum we use only the 2 labels data. Quantum simulation (Jax) models 83.87% have better accuracy than quantum hardware simulation 68.51%, classical 66.67%, and quantum real hardware ones 46.67%.

After we have described the accuracy results another metric that we should take into account is the training time of the models. We will follow the same structure that we use in the accuracy results.

First, we will focus on the comparative time between datasets. We note that the AD5 dataset is the one that takes more time to be trained at 38.8 min, followed by KAGG at 30.96 min, AD4 at 29.68 min, AD3 at 19.85 min, MILK at 19.52 min, and AD2 at 19.15 min. On average, classification models 28.32 min take more time than NER ones, 24.69 min to be trained.

Second, according to the number of labels of the models, the mean training time of 2 labels models is 9.4 min, meanwhile, the time of 3 labels is 29.21 min and 4 labels is 39.83 min.

[2] https://quantum-computing.ibm.com/services/resources?system=ibm_lagos

[3] https://quantum-computing.ibm.com/services/resources?system=ibm_oslo

[4] https://quantum-computing.ibm.com/services/resources?system=ibm_nairobi

**Table 2**
Model training results for 2 labels.

| Backend | Device | Ansatz | NLayers | Dataset AD2 | Dataset AD3 | Dataset AD4 | Dataset AD5 | Dataset MILK | Dataset KAGG |
|---|---|---|---|---|---|---|---|---|---|
| Classical | Classical device | – | – | 82.50 | 100.00 | 100.00 | 20.00 | 28.13 | 69.38 |
| Quantum-Jax | Classical device | IQP | 1 | 70.00 | 100.00 | 100.00 | 100.00 | 87.50 | **76.25** |
| Quantum-Jax | Classical device | IQP | 2 | **100.00** | **100.00** | **100.00** | **100.00** | 64.58 | 63.75 |
| Quantum-Sim | ibm_lagos | IQP | 1 | 60.00 | 90.00 | (-IQ-) | (-IQ-) | 65.63 | 58.75 |
| Quantum-Sim | ibm_lagos | IQP | 2 | 90.00 | 80.00 | (-IQ-) | (-IQ-) | 52.60 | 77.50 |
| Quantum-Real | ibm_lagos | IQP | 1 | 35.00 | (-ND-) | (-IQ-) | (-IQ-) | (-ND-) | (-ND-) |
| Quantum-Jax | Classical device | Sim14 | 1 | 100.00 | 100.00 | 100.00 | 100.00 | 65.62 | 51.25 |
| Quantum-Jax | Classical device | Sim14 | 2 | 45.00 | 100.00 | 100.00 | 90.00 | 69.79 | 42.50 |
| Quantum-Sim | ibm_lagos | Sim14 | 1 | 47.50 | 90.00 | (-IQ-) | (-IQ-) | 67.19 | 65.00 |
| Quantum-Sim | ibm_lagos | Sim14 | 2 | 50.00 | **100.00** | (-IQ-) | (-IQ-) | 68.75 | 45.63 |
| Quantum-Real | ibm_oslo | Sim14 | 1 | (-ND-) | (-ND-) | (-IQ-) | (-IQ-) | (-ND-) | 41.25 |
| Quantum-Jax | Classical device | Sim15 | 1 | 100.00 | 100.00 | 100.00 | 100.00 | 72.92 | 50.00 |
| Quantum-Jax | Classical device | Sim15 | 2 | 75.00 | 100.00 | 45.00 | 100.00 | 82.29 | 46.25 |
| Quantum-Sim | ibm_lagos | Sim15 | 1 | 75.00 | **100.00** | (-IQ-) | (-IQ-) | 67.19 | 49.38 |
| Quantum-Sim | ibm_lagos | Sim15 | 2 | 60.00 | 90.00 | (-IQ-) | (-IQ-) | 66.15 | 50.63 |
| Quantum-Real | ibm_nairobi | Sim15 | 1 | (-ND-) | (-ND-) | (-IQ-) | (-IQ-) | 60.42 | (-ND-) |
| Quantum-Jax | Classical device | Strong | 1 | 100.00 | 100.00 | 100.00 | 100.00 | 64.58 | 67.50 |
| Quantum-Jax | Classical device | Strong | 2 | 35.00 | 100.00 | 100.00 | 100.00 | **95.83** | 65.00 |
| Quantum-Sim | ibm_lagos | Strong | 1 | 52.50 | 52.50 | (-IQ-) | (-IQ-) | 89.58 | 61.25 |
| Quantum-Sim | ibm_lagos | Strong | 2 | 60.00 | **100.00** | (-IQ-) | (-IQ-) | 60.94 | 48.75 |
| Quantum-Real | ibm_lagos | Strong | 1 | (-ND-) | 50.00 | (-IQ-) | (-IQ-) | (-ND-) | (-ND-) |

**Table 3**
Model training accuracy results for 3 labels.

| Backend | Ansatz | NLayers | Dataset AD2 | Dataset AD3 | Dataset AD4 | Dataset AD5 | Dataset MILK | Dataset KAGG |
|---|---|---|---|---|---|---|---|---|
| Classical | – | – | **70.83** | 66.67 | **100.00** | **100.00** | **71.88** | **73.96** |
| Quantum-Jax | IQP | 1 | 30.00 | 66.67 | 46.67 | 66.67 | 52.78 | 39.17 |
| Quantum-Jax | IQP | 2 | **56.67** | **100.00** | (-ICR-) | (-ICR-) | 54.17 | **70.00** |
| Quantum-Jax | Sim14 | 1 | 30.00 | **100.00** | **86.67** | **96.66** | **69.44** | 68.33 |
| Quantum-Jax | Sim14 | 2 | 13.33 | 30.00 | 46.67 | 46.67 | 55.56 | 60.83 |
| Quantum-Jax | Sim15 | 1 | 10.00 | 66.67 | 36.67 | 66.67 | 33.33 | 30.83 |
| Quantum-Jax | Sim15 | 2 | 46.67 | 23.33 | 16.67 | 16.67 | 25.00 | 27.50 |
| Quantum-Jax | Strong | 1 | 46.67 | 66.67 | 76.67 | 66.67 | 61.11 | 64.17 |
| Quantum-Jax | Strong | 2 | 13.33 | 30.00 | 26.67 | 26.67 | 38.89 | 59.17 |

**Table 4**
Model training accuracy results for 4 labels.

| Backend | Ansatz | NLayers | Dataset AD2 | Dataset AD3 | Dataset AD4 | Dataset AD5 | Dataset MILK | Dataset KAGG |
|---|---|---|---|---|---|---|---|---|
| Classical | – | – | **87.50** | **100.00** | **100.00** | **100.00** | **72.81** | **70.57** |
| Quantum-Jax | IQP | 1 | 45.00 | 75.00 | 60.00 | 50.00 | 35.42 | 25.63 |
| Quantum-Jax | IQP | 2 | 50.00 | **100.00** | (-ICR-) | (-ICR-) | 45.83 | 26.25 |
| Quantum-Jax | Sim14 | 1 | 45.00 | **100.00** | 60.00 | 65.00 | 36.46 | 44.38 |
| Quantum-Jax | Sim14 | 2 | 30.00 | 32.50 | 32.50 | 35.00 | 45.83 | 48.75 |
| Quantum-Jax | Sim15 | 1 | 22.50 | 12.50 | 35.00 | 70.00 | 22.92 | 41.88 |
| Quantum-Jax | Sim15 | 2 | 30.00 | 47.50 | 35.00 | 37.50 | 37.50 | 23.12 |
| Quantum-Jax | Strong | 1 | 37.50 | 75.00 | 60.00 | 25.00 | 34.38 | 57.50 |
| Quantum-Jax | Strong | 2 | 30.00 | 42.50 | 40.00 | **100.00** | 37.50 | 46.88 |

Third, the comparative between *ansatz* (IQP, Sim14, Sim15 and Strong). We notice Sim14 *ansatz* 36.56 min takes more time than IQP *ansatz* 29.69 min, Sim15 *ansatz* 24.03 min and Strong *ansatz* 26.28.

Fourth, the models with 1 quantum layer of each *ansatz* are trained in 21.65 min, meanwhile the 2 quantum layers in 37.03 min.

To conclude, in the comparison between classical and quantum models in the datasets with 2 labels, we can observe that classical ones are trained in 1 min, meanwhile quantum simulation (Jax) takes 11 min, quantum hardware simulation in 577 min (10 h), and quantum real hardware in 7194 min (120 h).

## 5. Discussion

In this section, we are going to focus on the pipeline, limitations, and results obtained from the sentiment classification and NER-trained models in Section 4.

For the entire pipeline, we use in all of the models a combination of the libraries lambeq and DisCoPy. These are the chosen ones because they allow us to convert the sentences into pregroup grammars, transform them into *ansatz* and train the constructed models in different backends, including real quantum ones.

Due to the variety that these libraries offered, we try to combine backends (classical, quantum simulator, real quantum device simulator, real quantum device) with various types of *ansatz*, (IQP, Sim14, Sim15, and Strong) and configurations (1 or 2 layers of each *ansatz*) and we try to find the limits of the actual quantum devices in particular QNLP tasks.

One of the aspects we have to focus on is the limitations of the actual near-term quantum devices and the limitations of the classic hardware used in this paper. We have a 7 qubit limitation in real hardware (ibm_lagos, ibm_oslo, ibm_nairobi) because is the maximum qubits available in IBM open quantum systems. Also, we have a limitation in the RAM Memory used for the simulation of a quantum device with the Jax library, which is 32 GB.

We analyze the results using the same criteria we use in Section 4. According to the datasets, labels of each model, applied *ansatz*

**Table 5**
Model training time (min) for 2 labels — Quantum Real Devices.

| Backend | Device | Ansatz | NLayers | Dataset AD2 | Dataset AD3 | Dataset AD4 | Dataset AD5 | Dataset MILK | Dataset KAGG |
|---|---|---|---|---|---|---|---|---|---|
| Classical | Classical device | – | – | <1 | <1 | 1 | 1 | 1 | 1 |
| Quantum-Jax | Classical device | IQP | 1 | 2 | 2 | 6 | 8 | 5 | 4 |
| Quantum-Jax | Classical device | IQP | 2 | 6 | 6 | 23 | 31 | 9 | 6 |
| Quantum-Sim | ibm_lagos | IQP | 1 | 207 | 214 | (-IQ-) | (-IQ-) | 345 | 251 |
| Quantum-Sim | ibm_lagos | IQP | 2 | 371 | 365 | (-IQ-) | (-IQ-) | 527 | 720 |
| Quantum-Real | ibm_lagos | IQP | 1 | 4894 | (-ND-) | (-IQ-) | (-IQ-) | (-ND-) | (-ND-) |
| Quantum-Jax | Classical device | Sim14 | 1 | 6 | 6 | 9 | 12 | 14 | 12 |
| Quantum-Jax | Classical device | Sim14 | 2 | 11 | 10 | 17 | 23 | 25 | 19 |
| Quantum-Sim | ibm_lagos | Sim14 | 1 | 412 | 358 | (-IQ-) | (-IQ-) | 882 | 739 |
| Quantum-Sim | ibm_lagos | Sim14 | 2 | 662 | 612 | (-IQ-) | (-IQ-) | 1523 | 1174 |
| Quantum-Real | ibm_oslo | Sim14 | 1 | (-ND-) | (-ND-) | (-IQ-) | (-IQ-) | (-ND-) | 9759 |
| Quantum-Jax | Classical device | Sim15 | 1 | 4 | 4 | 7 | 9 | 10 | 8 |
| Quantum-Jax | Classical device | Sim15 | 2 | 7 | 7 | 12 | 15 | 17 | 13 |
| Quantum-Sim | ibm_lagos | Sim15 | 1 | 263 | 252 | (-IQ-) | (-IQ-) | 635 | 521 |
| Quantum-Sim | ibm_lagos | Sim15 | 2 | 463 | 417 | (-IQ-) | (-IQ-) | 1051 | 782 |
| Quantum-Real | ibm_nairobi | Sim15 | 1 | (-ND-) | (-ND-) | (-IQ-) | (-IQ-) | 8832 | (-ND-) |
| Quantum-Jax | Classical device | Strong | 1 | 4 | 4 | 7 | 8 | 11 | 9 |
| Quantum-Jax | Classical device | Strong | 2 | 8 | 7 | 12 | 16 | 19 | 14 |
| Quantum-Sim | ibm_lagos | Strong | 1 | 298 | 268 | (-IQ-) | (-IQ-) | 667 | 557 |
| Quantum-Sim | ibm_lagos | Strong | 2 | 505 | 439 | (-IQ-) | (-IQ-) | 1140 | 837 |
| Quantum-Real | ibm_lagos | Strong | 1 | (-ND-) | 5292 | (-IQ-) | (-IQ-) | (-ND-) | (-ND-) |

**Table 6**
Model training time (min) results for 3 labels.

| Backend | Ansatz | NLayers | Dataset AD2 | Dataset AD3 | Dataset AD4 | Dataset AD5 | Dataset MILK | Dataset KAGG |
|---|---|---|---|---|---|---|---|---|
| Classical | – | – | 1 | 1 | 1 | 2 | 1 | 1 |
| Quantum-Jax | IQP | 1 | 14 | 14 | 63 | 82 | 8 | 10 |
| Quantum-Jax | IQP | 2 | 63 | 62 | (-ICR-) | (-ICR-) | 26 | 28 |
| Quantum-Jax | Sim14 | 1 | 18 | 19 | 29 | 38 | 21 | 38 |
| Quantum-Jax | Sim14 | 2 | 36 | 36 | 58 | 75 | 39 | 72 |
| Quantum-Jax | Sim15 | 1 | 12 | 12 | 20 | 26 | 14 | 25 |
| Quantum-Jax | Sim15 | 2 | 23 | 23 | 38 | 51 | 25 | 44 |
| Quantum-Jax | Strong | 1 | 13 | 13 | 21 | 27 | 15 | 28 |
| Quantum-Jax | Strong | 2 | 26 | 26 | 43 | 56 | 29 | 53 |

**Table 7**
Model training time (min) results for 4 labels.

| Backend | Ansatz | NLayers | Dataset AD2 | Dataset AD3 | Dataset AD4 | Dataset AD5 | Dataset MILK | Dataset KAGG |
|---|---|---|---|---|---|---|---|---|
| Classical | – | – | 1 | 1 | 2 | 3 | 1 | 2 |
| Quantum-Jax | IQP | 1 | 20 | 20 | 84 | 110 | 12 | 15 |
| Quantum-Jax | IQP | 2 | 84 | 85 | (-ICR-) | (-ICR-) | 27 | 15 |
| Quantum-Jax | Sim14 | 1 | 25 | 25 | 40 | 54 | 29 | 57 |
| Quantum-Jax | Sim14 | 2 | 49 | 50 | 80 | 103 | 54 | 107 |
| Quantum-Jax | Sim15 | 1 | 17 | 17 | 27 | 35 | 19 | 36 |
| Quantum-Jax | Sim15 | 2 | 32 | 32 | 53 | 70 | 35 | 66 |
| Quantum-Jax | Strong | 1 | 18 | 18 | 29 | 37 | 21 | 43 |
| Quantum-Jax | Strong | 2 | 17 | 36 | 60 | 78 | 40 | 80 |

**Table 8**
Mean grouped metrics.

| | | Accuracy | Time |
|---|---|---|---|
| Labels | 2 | 81,96 | 8,42 |
| | 3 | 52,83 | 29,21 |
| | 4 | 50,41 | 39,83 |
| Layers | 1 | 64,12 | 21,65 |
| | 2 | 57,85 | 37,03 |
| Ansatz | IQP | 67,50 | 29,69 |
| | Sim14 | 62,65 | 36,56 |
| | Sim15 | 53,46 | 24,03 |
| | Strong | 61,48 | 26,28 |

(IQP, Sim14, Sim15, Strong), the number of layers of each *ansatz*, and classical and quantum hardware.

**Table 9**
2 labels mean comparison.

| | Accuracy | Time |
|---|---|---|
| Classical | 66,67 | 1 |
| Quantum-Jax | 83,87 | 11 |
| Quantum-Sim | 68,51 | 577 |
| Quantum-Real | 46,67 | 7194 |

The significantly less accuracy in NER datasets against classification ones can be explained because the satisfactory first experiments carried out with lambeq are classification tasks, no name entity recognition tasks. One solution can be to create or implement new *ansatz* that adjust better the NER parameters in the model.

First, related to the number of labels of the models, in the models with 3 and 4 labels, we achieve the worst accuracy, this can be due to

the increase in complexity of the models because the required qubits used to store them increase from 1 qubit to 2 qubits.

Second, the IQP *ansatz* model is the first one implemented in lambeq and is the *ansatz* in which the developers tested the first satisfactory results. We do not know yet how the capabilities of the *ansatz* affect each model. However, a future line of this paper is to investigate how an *ansatz* can adjust better or not the model parameters.

Third, the models with 1 quantum layer have greater accuracy than models with 2 layers, so we can conclude that the complexity of the *ansatz* is not correlated with a superior model in accuracy terms.

Fourth, classical and quantum models in datasets with 2 labels, we observe that quantum simulation (Jax) and quantum hardware simulation obtained better mean results than classical ones. However, the time to train this quantum models x10 and x500 times greater than classical ones. Also, we observe that quantum real hardware obtained the worst accuracy, this may be because quantum real devices are still imperfect and are not fault-tolerant, they cannot achieve the same results as the simulators, which are theoretically perfect. Regarding datasets with 3 labels quantum simulation (Jax) models can be comparable to classical ones even can obtained better results in one dataset. In the datasets with 4 labels classical ones obtained greater results than quantum simulation (Jax) models.

To conclude, the training time of the models is greater when the datasets or *ansatz* are more complex. Also, the training time in quantum real devices is larger due to the queue we have to wait to train them for the quantum resources. Even though the accuracy results are in some cases comparable between classical and quantum, the time to train the model is huge compared with the classical ones.

## 6. Conclusions and future lines

Through this experiment, it can be concluded that in some cases, the quantum models can be comparable with classical models in the simpler classification tasks, because they can obtain the same or better results in some datasets. When the number of labels or the complexity of sentences increases, the accuracy is not higher enough to provide a successful model, or we face issues with the computational resources in the model training. The computational resources problems are even present when we simulate the quantum computers in classical machines – lack of CUDA/RAM memory – or when we perform the task in real quantum computers – too much training time –. When the resources' computational problems are solved, we will be able to train the models using complex *ansatz* or training the models efficiently, which will help to solve the accuracy problems in the complex datasets.

For future lines of work, it is proposed to expand the number of words that can be included in the same sentence, increase the connection complexity between words, apply the models to different languages, or create a model to join different sentences. Also, to be able to perform these higher complex tasks by simulating the quantum devices or in the actual near-term devices, which nowadays is not possible. Another subfield in QNLP that is nearly unexplored is the generation of text (Karamlou et al., 2022).

## CRediT authorship contribution statement

**David Peral-García:** Conceptualization, Methodology, Writing – original draft. **Juan Cruz-Benito:** Writing – review & editing, Supervision. **Francisco José García-Peñalvo:** Writing – review & editing, Supervision.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## Appendix A. Implementation

In this section we add some code examples to give an explanation of their proposed model. The complete code is available in https://github.com/DaveHub5/QNLP_Classification_2_3_4_labels.

### A.1. Classical computing

First, we parse the sentences into string diagrams.

```
from lambeq import BobcatParser

parser = BobcatParser(verbose='text')

raw_train_diagrams =
    parser.sentences2diagrams(train_data)
```

Secondly, we encoded the diagram into an ansatz, in this case, we use Spider ansatz.

```
from discopy import Dim
from lambeq import AtomicType, SpiderAnsatz

ansatz = SpiderAnsatz({AtomicType.NOUN: Dim(2),
                       AtomicType.SENTENCE: Dim(2)})

train_circuits = [ansatz(diagram) for diagram in
    train_diagrams]
```

Last, a Pytorch model is trained.

```
from lambeq import PytorchTrainer

trainer = PytorchTrainer(
      model=model,
      loss_function=torch.nn.BCEWithLogitsLoss(),
      optimizer=torch.optim.AdamW,
      learning_rate=LEARNING_RATE,
      epochs=EPOCHS,
      evaluate_functions=eval_metrics,
      evaluate_on_train=True,
      verbose='text',
      seed=SEED)

trainer.fit(train_dataset, val_dataset,
    evaluation_step=1, logging_step=5)
```

### A.2. Quantum computing

First, we parse the sentences into string diagrams.

```
from lambeq import BobcatParser

parser = BobcatParser(verbose='text')

raw_train_diagrams =
    parser.sentences2diagrams(train_data)
```

Secondly, we encoded the diagram into an ansatz, in this case, we use IQP ansatz.

```
from lambeq import AtomicType, IQPAnsatz, remove_cups

ansatz = IQPAnsatz({AtomicType.NOUN: 1,
    AtomicType.SENTENCE: 1},
                n_layers=1, n_single_qubit_params=3)

train_circuits = [ansatz(remove_cups(diagram)) for
    diagram in train_diagrams]
```

Third, the quantum simulator backend is defined.

```
from pytket.extensions.qiskit import AerBackend
from lambeq import TketModel

all_circuits = train_circuits + val_circuits

backend = AerBackend()
backend_config = {
    'backend': backend,
    'compilation': backend.default_compilation_pass(2),
    'shots': 4096
}

model = TketModel.from_diagrams(all_circuits,
    backend_config=backend_config)
```

Finally, a quantum model is trained

```
from lambeq import QuantumTrainer, SPSAOptimizer

trainer = QuantumTrainer(
    model,
    loss_function=bce,
    epochs=EPOCHS,
    optimizer=SPSAOptimizer,
    optim_hyperparams={'a': 0.02, 'c': 0.03,
        'A':0.01*EPOCHS},
    evaluate_functions=eval_metrics,
    evaluate_on_train=True,
    verbose = 'text',
    seed=0
)

trainer.fit(train_dataset, val_dataset,
    evaluation_step=1, logging_step=12)
```

## References


Bauer, B., Bravyi, S., Motta, M., & Chan, G. K.-L. (2020). Quantum algorithms for quantum chemistry and quantum materials science. *Chemical Reviews*, *120*(22), 12685–12717. http://dx.doi.org/10.1021/acs.chemrev.9b00829, PMID: 33090772.

Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., & Lloyd, S. (2017). Quantum machine learning. *Nature*, *549*(7671), 195–202. http://dx.doi.org/10.1038/nature23474.

Coecke, B., de Felice, G., Meichanetzidis, K., & Toumi, A. (2020). Foundations for near-term quantum natural language processing.

Coecke, B., Sadrzadeh, M., & Clark, S. (2010). Mathematical foundations for a compositional distributional model of meaning. http://dx.doi.org/10.48550/ARXIV.1003.4394.

de Felice, G., Toumi, A., & Coecke, B. (2021). DisCoPy: Monoidal categories in python. *Electronic Proceedings in Theoretical Computer Science*, *333*, 183–197. http://dx.doi.org/10.4204/eptcs.333.13.

Ding, C., Di Federico, M., Hatridge, M., Houck, A., Leger, S., Martinez, J., Miao, C., Schuster, D. I., Stefanazzi, L., Stoughton, C., Sussman, S., Treptow, K., Uemura, S., Wilcer, N., Zhang, H., Zhou, C., & Cancelo, G. (2023). Experimental advances with the QICK (quantum instrumentation control kit) for superconducting quantum hardware.

Emmanoulopoulos, D., & Dimoska, S. (2022). Quantum machine learning in finance: Time series forecasting. http://dx.doi.org/10.48550/ARXIV.2202.00599.

García, D. P., Cruz-Benito, J., & García-Peñalvo, F. J. (2022). Systematic literature review: Quantum machine learning and its applications.

Gokhale, A., Pande, M. B., & Pramod, D. (2020). Implementation of a quantum transfer learning approach to image splicing detection. *International Journal of Quantum Information*, *18*(5), http://dx.doi.org/10.1142/S0219749920500240.

Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on theory of computing* (pp. 212–219). New York, NY, USA: Association for Computing Machinery, http://dx.doi.org/10.1145/237814.237866.

Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M., & Gambetta, J. M. (2019). Supervised learning with quantum-enhanced feature spaces. *Nature*, *567*(7747), 209–212. http://dx.doi.org/10.1038/s41586-019-0980-2.

Karamlou, A., Pfaffhauser, M., & Wootton, J. (2022). Quantum natural language generation on near-term devices. http://dx.doi.org/10.48550/ARXIV.2211.00727.

Kartsaklis, D., Fan, I., Yeung, R., Pearson, A., Lorenz, R., Toumi, A., de Felice, G., Meichanetzidis, K., Clark, S., & Coecke, B. (2021). Lambeq: An efficient high-level Python library for quantum NLP.

Kerenidis, I., & Prakash, A. (2017). Quantum recommendation systems. In C. H. Papadimitriou (Ed.), *Leibniz international proceedings in informatics (lIPIcs)*: *67*, *8th Innovations in theoretical computer science conference* (pp. 49:1–49:21). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, http://dx.doi.org/10.4230/LIPIcs.ITCS.2017.49.

Kissinger, A., & van de Wetering, J. (2020). PyZX: Large Scale Automated Diagrammatic Reasoning. In B. Coecke, & M. Leifer (Eds.), *Electronic proceedings in theoretical computer science*: *vol. 318*, proceedings 16th international conference on *quantum physics and logic*, chapman university, orange, CA, USA., 10-14 June 2019 (pp. 229–241). Open Publishing Association, http://dx.doi.org/10.4204/EPTCS.318.14.

Lambek, J. (1958). The mathematics of sentence structure. *American Mathematical Monthly*, *65*(3), 154–170. http://dx.doi.org/10.1080/00029890.1958.11989160.

Lorenz, R., Pearson, A., Meichanetzidis, K., Kartsaklis, D., & Coecke, B. (2023). QNLP in practice: Running compositional models of meaning on a quantum computer. *Journal of Artificial Intelligence Research*, *76*, 1305–1342. http://dx.doi.org/10.1613/jair.1.14329.

Lukac, M., Abdiyeva, K., & Kameyama, M. (2018). CNOT-measure quantum neural networks. In *Proceedings of the international symposium on multiple-valued logic, vol. 2018-May* (pp. 186–191). IEEE Computer Society, http://dx.doi.org/10.1109/ISMVL.2018.00040.

Romito, A. (2023). Quantum hardware measures up to the challenge. *Nature Physics*, *19*(9), 1234–1235. http://dx.doi.org/10.1038/s41567-023-02090-8.

Schuld, M., Bocharov, A., Svore, K. M., & Wiebe, N. (2020). Circuit-centric quantum classifiers. *Physical Review A*, *101*(3), http://dx.doi.org/10.1103/physreva.101.032308.

Shenoy, K. S., Sheth, D. Y., Behera, B. K., & Panigrahi, P. K. (2020). Demonstration of a measurement-based adaptation protocol with quantum reinforcement learning on the IBM q experience platform. *Quantum Information Processing*, *19*(5), http://dx.doi.org/10.1007/s11128-020-02657-x.

Shlosberg, A., Jena, A. J., Mukhopadhyay, P., Haase, J. F., Leditzky, F., & Dellantonio, L. (2023). Adaptive estimation of quantum observables. *Quantum*, *7*, 906. http://dx.doi.org/10.22331/q-2023-01-26-906.

Shor, P. W. (1997). Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, *26*(5), 1484–1509. http://dx.doi.org/10.1137/s0097539795293172.

Sim, S., Johnson, P. D., & Aspuru-Guzik, A. (2019). Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms. *Advanced Quantum Technologies*, *2*(12), Article 1900070. http://dx.doi.org/10.1002/qute.201900070.

Toumi, A., de Felice, G., & Yeung, R. (2022). DisCoPy for the quantum computer scientist. http://dx.doi.org/10.48550/ARXIV.2205.05190.

Widdows, D., Alexander, A., Zhu, D., Zimmerman, C., & Majumder, A. (2024). Near-term advances in quantum natural language processing. *Annals of Mathematics and Artificial Intelligence*, http://dx.doi.org/10.1007/s10472-024-09940-y.

Wu, S. L., Sun, S., Guan, W., Zhou, C., Chan, J., Cheng, C. L., Pham, T., Qian, Y., Wang, A. Z., Zhang, R., Livny, M., Glick, J., Barkoutsos, P. K., Woerner, S., Tavernelli, I., Carminati, F., Di Meglio, A., Li, A. C. Y., Lykken, J., .... Wei, T.-C. (2021). Application of quantum machine learning using the quantum kernel algorithm on high energy physics analysis at the LHC. arXiv:2104.05059.